Try

Try · Parse config int with recovery · Build with Waqar

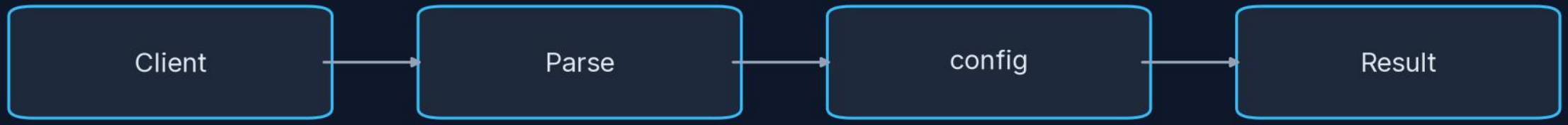
Today's Hook

- Problem: Parse config int with recovery
- Pattern: Try — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Parse config int with recovery
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Try: Parse config int with recovery

Real-World Scenario

- Running example: Parse config int with recovery
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.try_.problem;

import java.util.*;

public final class NaiveCfg {
  public static int port(String s){ return Integer.parseInt(s); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.try_.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Try<A> permits Success, Failure { A getOrElse(A fallback); boolean isSuccess(); }
record Success<A>(A value) implements Try<A> { public A getOrElse(A f){return value;} public boolean isSuccess(){return true;} }
record Failure<A>(Throwable error) implements Try<A> { public A getOrElse(A f){return f;} public boolean isSuccess(){return false;} }
final class Tries {
  static <A> Try<A> of(Callable<A> c){ try { return new Success<>(c.call()); } catch(Exception e){ return new Failure<>(e); } }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.try_.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Try: parse config int with recovery ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveCfg.port("x")); } catch(Exception e){ System.out.println("caught"); }
System.out.println("--- try ---");
System.out.println(Tries.of(() -> Integer.parseInt("8080")).getOrElse(-1));
System.out.println(Tries.of(() -> Integer.parseInt("x")).getOrElse(8080));
  }
}
```

Walkthrough Output

```
=== Try: parse config int with recovery ===
--- naive ---
caught
--- try ---
8080
8080
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Try.

Related Patterns

- Contrast Try with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Try → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.